

SuperKart Sales Forecasting

SuperKart: Model Deployment

This file is meant for personal use by cmurray4492@gmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- EDA Results
- Data Preprocessing
- Model Building
- Model Performance Summary
- Deployment

Executive Summary

- We can use this forecasting model to predict the total sales that will be done by SuperKart in the next quarter.
- The model explains around 90% of the variation in the data and can predict within ~6.5% of the actual sales of the product.
- OUT004, which is of Supermarket Type2, located in a Tier 2 city and having store size as medium, is performing well. SuperKart can look to increase the size of this store from medium to high. They can also look to set up stores in this type of city having comparable socio-economic conditions in order to expand their business and reach.
- OUT002, being a food mart in a Tier 3 city and having small store size, is also performing well. SuperKart can look to upgrade its size or target similar cities for business expansion.
- OUT001 which is a store of Supermarket Type 1, located in a Tier 2 city and having store size as high is not performing upto the mark. SuperKart can look to look build new marketing strategies (maybe give attractive discounts and offers) in this store in order to attract more customers.

This file is meant for personal use by cmurray4492@gmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Executive Summary

- Similar approach can be taken to increase the business of OUT003 which is a Departmental store in a Tier 1 city and having store size as medium. It is the premium store of the company where most of the costly goods are sold, so the correct set of audience should be targeted.
- Daily needs like fruits and vegetables and snack foods are the biggest contributors to the revenue across all the stores. So SuperKart should look to maintain the inventory of these products properly and ensure that these products never face a shortage.
- Customers' details like age and gender can be incorporated in this model so that the company gets to know their target audience well and can build their sales strategies according to that.
- The company should also keep a watch for the number of festive occasions present in a quarter so that they can strategize their inventory accordingly.

Business Problem Overview

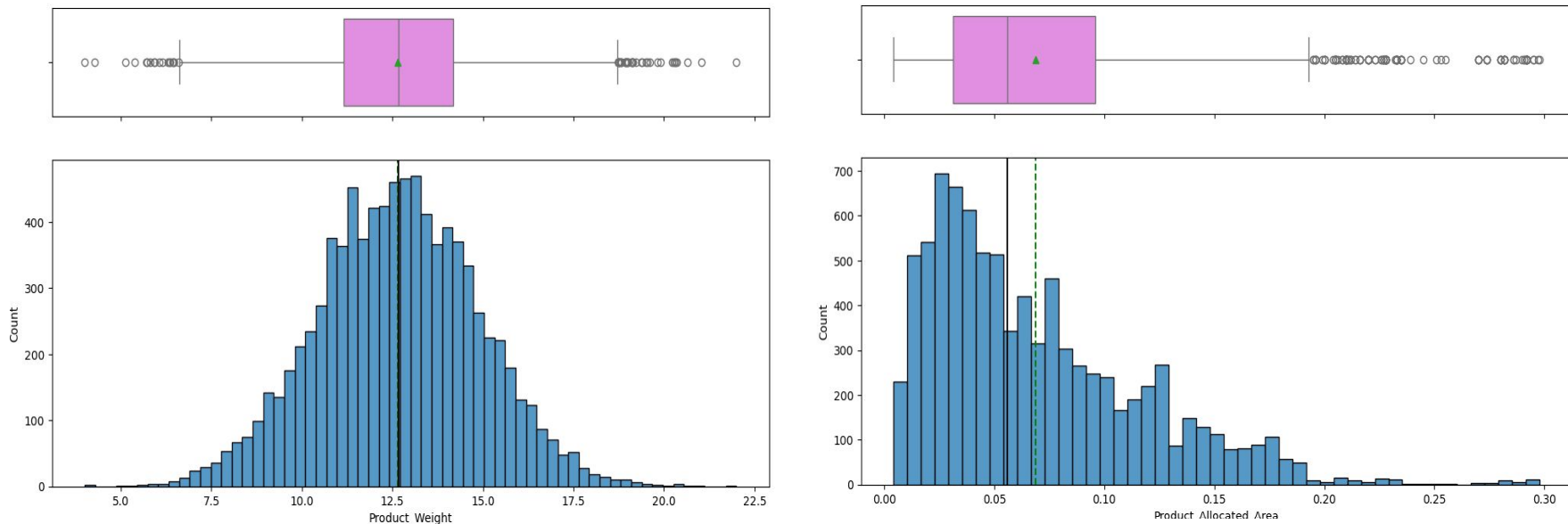
- A sales forecast is a prediction of future sales revenue based on historical data, industry trends, and the status of the current sales pipeline.
 - Businesses use the sales forecast to estimate weekly, monthly, quarterly, and annual sales totals.
- A company needs to make an accurate sales forecast as it adds value across an organization and helps the different verticals to chalk out their future course of action.
- Forecasting helps an organization plan its sales operations by region and provides valuable insights to the supply chain team regarding the procurement of goods and materials.
 - An accurate sales forecast process has many benefits which include improved decision-making about the future and reduction of sales pipeline and forecast risks.
 - Moreover, it helps to reduce the time spent in planning territory coverage and establish benchmarks that can be used to assess trends in the future.

Solution Approach

1. **Define the business problem** of forecasting sales revenue for SuperKart to optimize inventory and sales strategies.
2. **Explore and preprocess the data**, including EDA, feature engineering (like store age and product category), and handling categorical variables.
3. **Build and evaluate various regression models** (e.g., Decision Tree, Random Forest, XGBoost) and improve performance through hyperparameter tuning using cross-validation.
4. **Deploy the best-performing model** as a backend API using Flask and containerize it with Docker, and create a user-friendly frontend using Streamlit.
5. **Generate actionable business insights** and recommendations based on the analysis and model results, such as store performance and popular product categories.

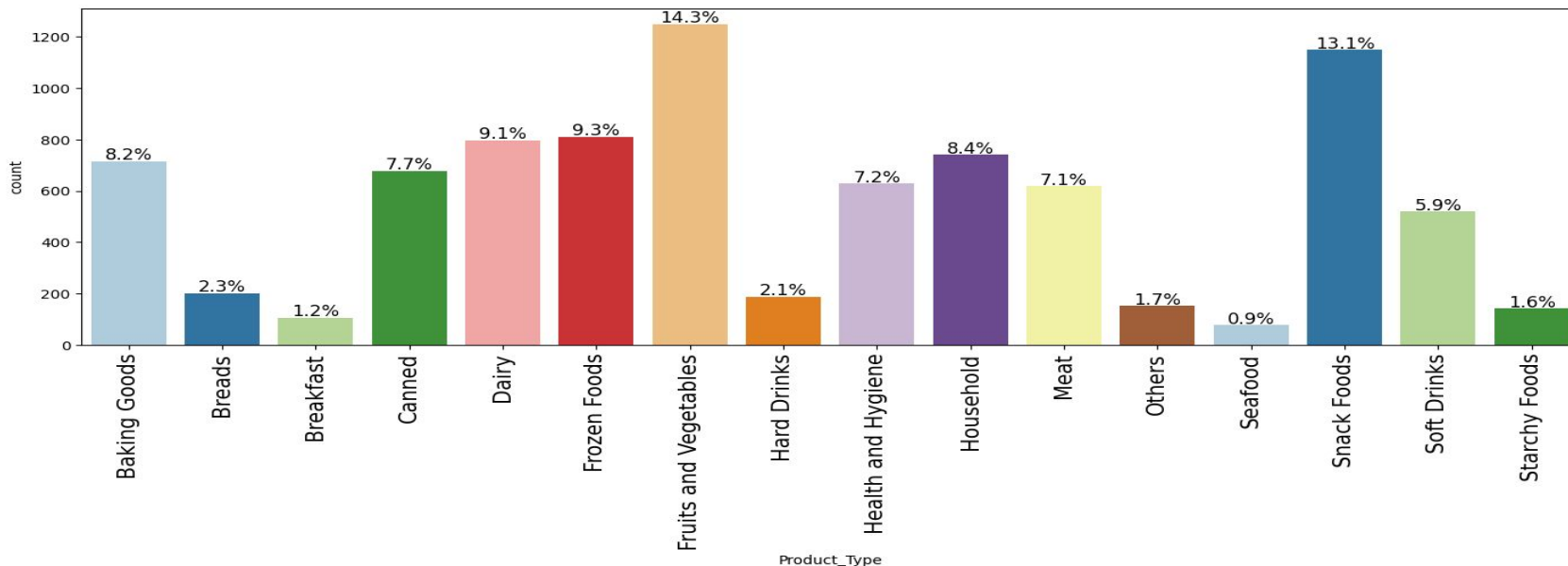
EDA Results

- The product weight and **Product MRP** are normally distributed, while the **Product Allocated Area** shows a right-skewed distribution.



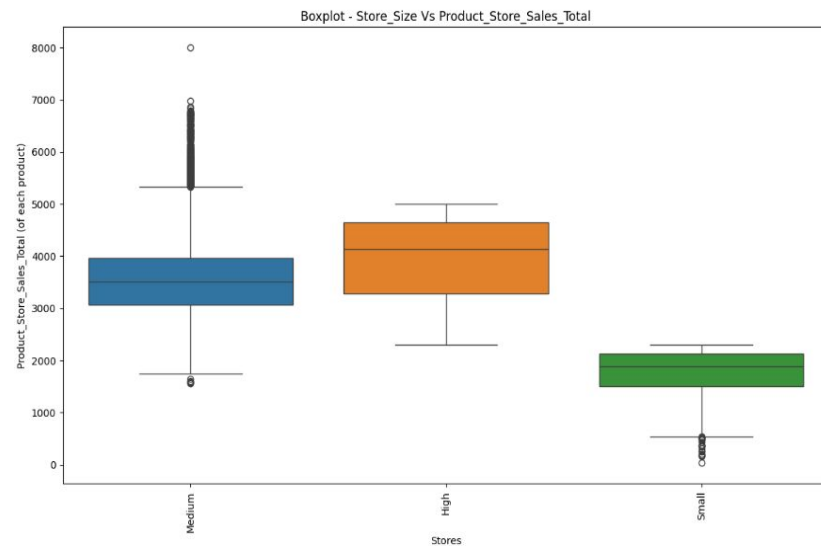
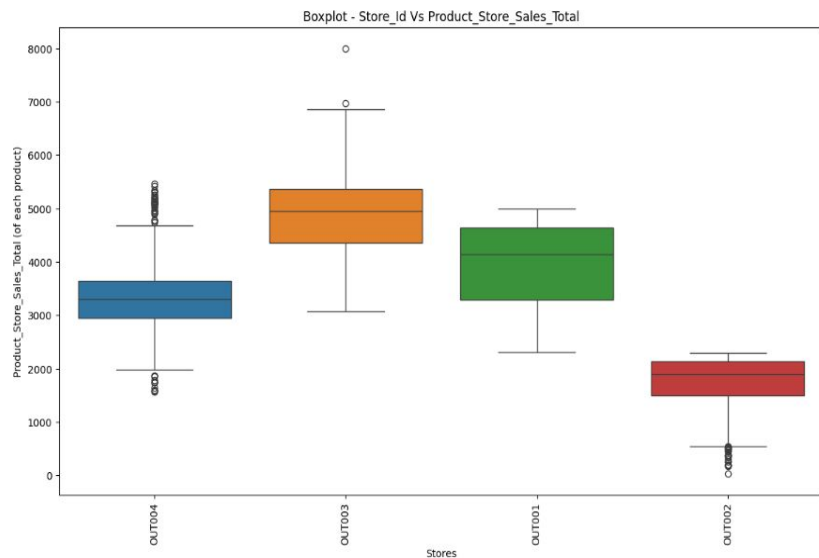
EDA Results

- Fruits and vegetables (~14%) and Snack Foods (~13%) were the most frequently purchased product types across all stores, and they are also the biggest contributors to the overall revenue



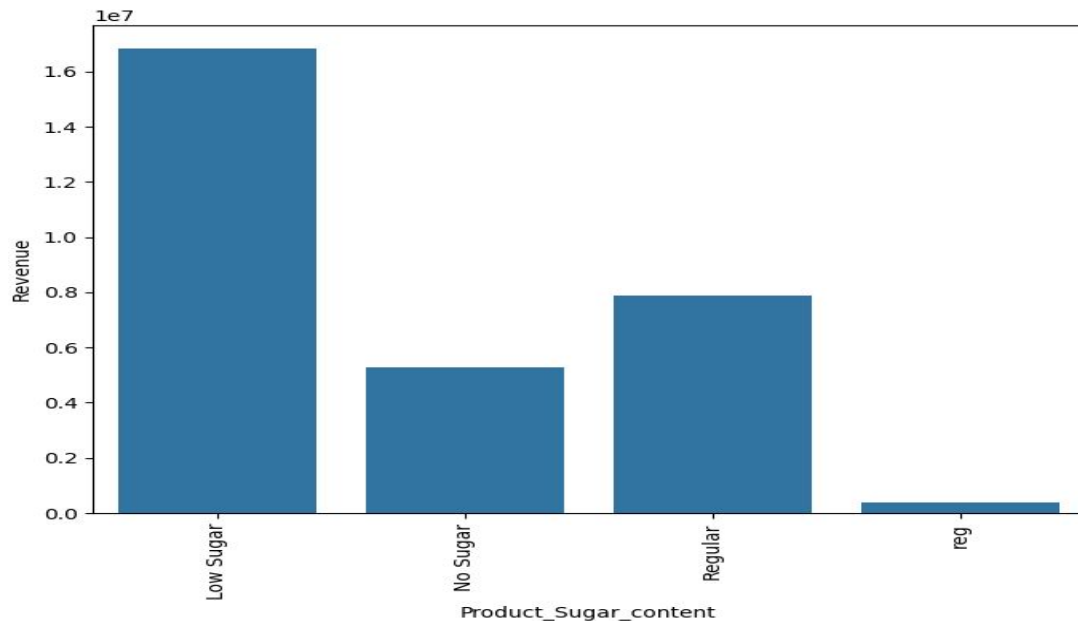
EDA Results

- OUT004 (Supermarket Type 2 in a Tier 2 city, medium size) generated the highest total revenue, significantly more than the other three stores. Medium-sized stores and stores located in Tier 2 cities contribute the most to the total revenue



EDA Results

- Low sugar content products constitute the highest percentage of products sold and are the biggest contributors to the revenue



Data Preprocessing

Feature Engineering

- A new feature, **Product_Id_char**, was created by extracting the first two characters from the **Product_Id** column. This was done to identify product categories, as observed with examples like "FD" for food items and "DR" for drinks.
- Another feature, **Store_Age_Years**, was calculated by subtracting the **Store_Establishment_Year** from the current year (assumed to be 2025 in the code). This aimed to incorporate the duration a store has been in business, potentially influencing customer trust or infrastructure.
- A broad **Product_Type_Category** was created by grouping the 16 original **Product_Type** categories into two broader categories: "Perishables" and "Non Perishables". This was done to reduce the number of product type categories, with specific types like "Dairy", "Meat", and "Fruits and Vegetables" being classified as "Perishables"

Data Preprocessing

Outlier Detection

- Outliers were detected using boxplots for the numerical columns: **Product_Weight**, **Product_Allocated_Area**, **Product_MRP**, and **Product_Store_Sales_Total**. **Store_Establishment_Year** and the newly created **Store_Age_Years** were excluded from this check.
- The observations from the boxplots indicated the presence of "quite a few outliers" in the data.
- However, these outliers were not treated because they were considered "proper values"

Data Preprocessing

Data Preparation for Modeling

- The initial step involved dropping unnecessary columns that were deemed not required for modeling. These columns were **Product_Id**, **Product_Type**, **Store_Id**, and **Store_Establishment_Year**.
- The dataset was then split into features (X) and the target variable (y), where **Product_Store_Sales_Total** was the target.
- Finally, the data was split into training and testing sets using a 70:30 ratio, with `random_state=1` for reproducibility and `shuffle=True` to ensure randomness in the split

Data Preprocessing

Preprocessing Pipeline:

- A preprocessing pipeline was created using **make_column_transformer** from `sklearn.compose`.
- The pipeline was specifically designed to handle categorical features, those with object or category data types.
- The pipeline consists of a Pipeline object that includes a **OneHotEncoder** to convert the categorical features into a numerical format suitable for machine learning models.
 - The **handle_unknown='ignore'** parameter in **OneHotEncoder** ensures that unknown categories encountered during prediction will be ignored rather than throwing an error.
- This preprocessor was later integrated into the model pipelines using **make_pipeline**, combining the preprocessing steps with the model training

Model Building

- We'll fit different models on the train data and observe their performance.
- We'll try to improve that performance by tuning some hyperparameters available for that algorithm.
- We'll use GridSearchCv for hyperparameter tuning and **r2_score** to optimize the model.
 - **r2_score**, or R-squared, is the coefficient of determination is used to evaluate the performance of a regression model.
 - It is the amount of the variation in the output dependent attribute which is predictable from the input independent variables.

Model Building

- We start by building multiple ML models.
- The models built and their performance score are provided below

	XGboost	Gradient Boosting
Training Set R-square	0.69	0.69
Test Set R-square	0.67	0.66

- The XGboost model explains ~67–69% of the target variability; improvement is possible.
- The Gradient boosting model explains ~66–69% of the target variability; improvement is possible.

Model Improvement - Hyperparameter Tuning

- Grid Search with cross-validation was used for hyperparameter tuning
- The hyperparameter space checked and best hyperparameters obtained are as follows:

Hyperparameter	n_estimators	subsample	gamma	colsample_bytree	colsample_bylevel
Hyperparameter Space	75,100,125,150	0.7,0.8,0.9,1	0,1,3,5	0.7,0.8,0.9,1	0.7,0.8,0.9,1
Best Value Obtained	100	0.7	1	0.8	0.7

- The performance of the tuned XGboost model is as follows:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
Training Set Performance	597	469	0.69	0.825	0.17
Test Set Performance	616	485	0.67	0.67	0.18

Model Improvement - Hyperparameter Tuning

- Grid Search with cross-validation was used for hyperparameter tuning
- The hyperparameter space checked and best hyperparameters obtained are as follows:

Hyperparameter	n_estimators	subsample	max_features	max_depth
Hyperparameter Space	50,75,100,125,150,175	0.7,0.8,0.9,1	0.7,0.8,0.9,1	3,5,7,10
Best Value Obtained	100	0.7	0.7	5

- The performance of the tuned Gradient Boosting model is as follows:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
Training Set Performance	597	469	0.68	0.68	0.16
Test Set Performance	616	485	0.66	0.66	0.18

Model Performance Comparison

Training Performance

	XGboost	Tuned XGboost	Gradient Boosting	Tuned Gradient Boosting
RMSE	597	597	597	597
MAE	469	469	469	469
R-squared	0.68	0.69	0.68	0.68
Adj. R-squared	0.69	0.69	0.68	0.68
MAPE	0.17	0.17	0.17	0.17

Model Performance Comparison

Test Performance

	XGboost	Tuned XGboost	Gradient Boosting	Tuned Gradient Boosting
RMSE	616	616	616	616
MAE	485	485	485	485
R-squared	0.67	0.67	0.67	0.67
Adj. R-squared	0.67	0.67	0.66	0.67
MAPE	0.18	0.19	0.18	0.19

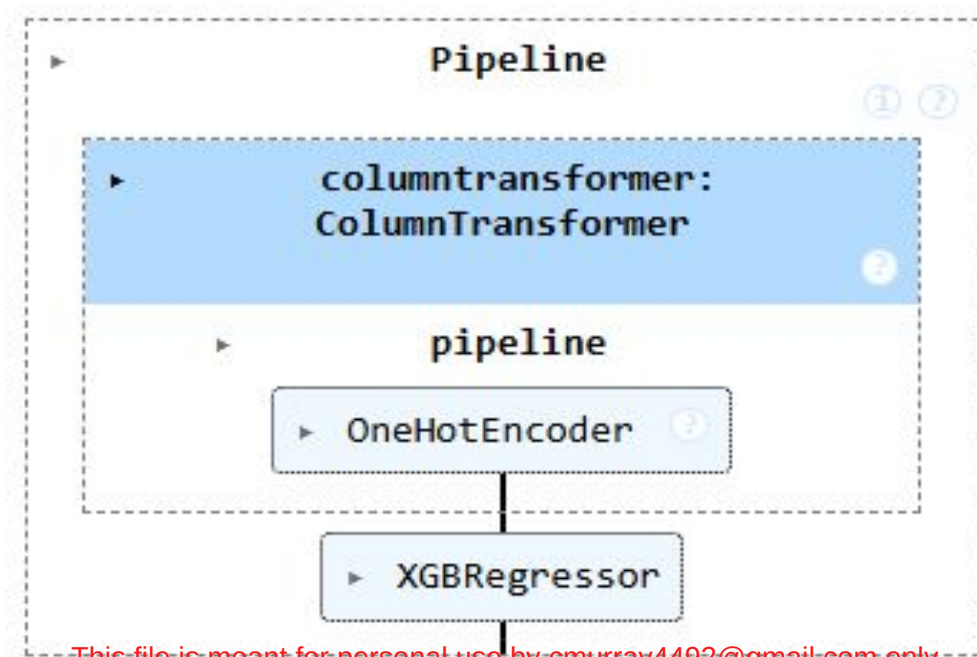
Final Model Selection

- All the tuned models show similar performance on both the training and test datasets.
- Since XGBoost is widely adopted, we will select the tuned XGBoost model as the best performing one.
- The test set performance of the tuned XGboost model is as follows:

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
Test Set Performance	616	485	0.67	0.67	0.19

Serialization

- The entire pipeline is is serialized which contains the OneHotEncoder and the XGBRegressor



This file is meant for personal use by cmurray4492@gmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Deployment - Backend

Default route



Deployment - Frontend

Spaces

like 0

Running

Logs

App

Files

Community

Settings

SuperKart System

Product Weight

12.66

-

+

Product Sugar Content

Low Sugar

▼

Product Allocated Area

0.03

-

+

Product MRP

117.08

-

+

Store Size

Small

▼

Store Location City Type

Tier 1

▼

Store Type

Supermarket Type1

▼

Product ID Character

FD

▼

Store Age (Years)

16

-

+

Product Type Category

Perishables

▼

Predict

Predicted Product Store Sales Total: ₹2685.43

This file is meant for personal use by cmurray4492@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

APPENDIX

This file is meant for personal use by cmurray4492@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Data Background and Contents

Attribute	Description
Product_Id	Unique identifier of each product, each identifier having two letters at the beginning followed by a number.
Product_Weight	Weight of each product
Product_Sugar_Content	Sugar content of each product like low sugar, regular and no sugar
Product_Allocated_Area	Ratio of the allocated display area of each product to the total display area of all the products in a store

Data Background and Contents

Attribute	Description
Product_Type	Broad category for each product like meat, snack foods, hard drinks, dairy, canned, soft drinks, health and hygiene, baking goods, bread, breakfast, frozen foods, fruits and vegetables, household, seafood, starchy foods, others
Product_MRP	Maximum retail price of each product
Store_Id	Unique identifier of each store
Store_Establishment_Year	Year in which the store was established
Store_Size	Size of the store depending on sq. feet like high, medium and low

This file is meant for personal use by cmurray4492@gmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Data Background and Contents

Attribute	Description
Store_Location_City_Type	Type of city in which the store is located like Tier 1, Tier 2 and Tier 3. Tier 1 consists of cities where the standard of living is comparatively higher than its Tier 2 and Tier 3 counterparts.
Store_Type	Type of store depending on the products that are being sold there like Departmental Store, Supermarket Type 1, Supermarket Type 2 and Food Mart
Product_Store_Sales_Total	Total revenue generated by the sale of that particular product in that particular store

This file is meant for personal use by cmurray4492@gmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Deployment - Backend (Flask API)

Hugging Face Search models, datasets, users...

Models Datasets Spaces Posts Docs Enterprise Pricing

Spaces: 0 Running Logs

App Files Community Settings

main SuperKart_API / app.py

Upload folder using huggingface_hub fd0bf2 VERIFIED 7 days ago

raw Copy download link History blame edit delete Safe 1.65 kB

```

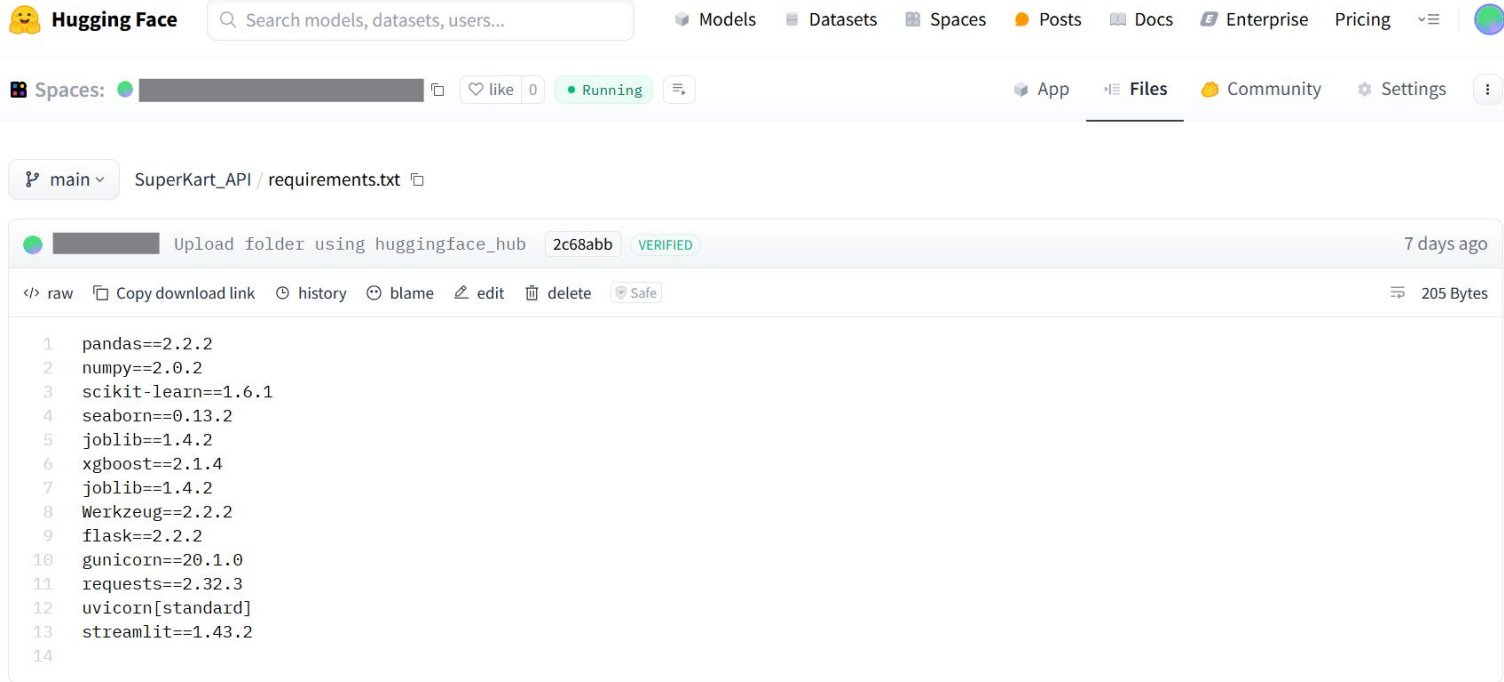
1
2 # Import necessary libraries
3 import numpy as np
4 import joblib # For loading the serialized model
5 import pandas as pd # For data manipulation
6 from flask import Flask, request, jsonify # For creating the Flask API
7
8 # Initialize Flask app with a name
9 superkart_api = Flask("SuperKart")
10
11 # Load the trained churn prediction model
12 model = joblib.load("superkart_model.joblib")
13
14 # Define a route for the home page
15 @superkart_api.get('/')
16 def home():
17     return "Welcome to the SuperKart System"
18
19 # Define an endpoint to predict churn for a single customer
20 @superkart_api.post('/v1/predict')
21 def predict_sales():
22     # Get JSON data from the request
23     data = request.get_json()
24
25     # Extract relevant customer features from the input data
26     sample = {
27         'Product_Weight': data['Product_Weight'],
28         'Product_Sugar_Content': data['Product_Sugar_Content'],
29         'Product_Allocated_Area': data['Product_Allocated_Area'],
30         'Product_MRP': data['Product_MRP'],
31         'Store_Size': data['Store_Size'],
32         'Store_Location_City_Type': data['Store_Location_City_Type'],
33         'Store_Type': data['Store_Type'],
34         'Product_Id_char': data['Product_Id_char'],
35         'Store_Age_Years': data['Store_Age_Years'],
36         'Product_Type_Category': data['Product_Type_Category']
37     }
38
39     # Convert the extracted data into a DataFrame
40     input_data = pd.DataFrame([sample])
41
42     # Make a churn prediction using the trained model
43     prediction = model.predict(input_data).tolist()[0]
44
45     # Return the prediction as a JSON response
46     return jsonify({'Sales': prediction})
47
48
49 # Run the Flask app in debug mode
50 if __name__ == '__main__':
51     superkart_api.run(debug=True)
52

```

This file is meant for personal use by cmurray4492@gmail.com only.

Sharing or publishing the contents in part or full is liable for legal action.
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Deployment - Backend (Flask API - requirements.txt)



The screenshot shows the Hugging Face interface. At the top, there's a search bar and navigation links for Models, Datasets, Spaces, Posts, Docs, Enterprise, Pricing, and a user profile. Below this, the 'Spaces' section is active, showing a space named 'SuperKart_API' with a 'Running' status. The 'Files' tab is selected, displaying a file named 'requirements.txt' (205 Bytes) uploaded 7 days ago. The file content is as follows:

```
1 pandas==2.2.2
2 numpy==2.0.2
3 scikit-learn==1.6.1
4 seaborn==0.13.2
5 joblib==1.4.2
6 xgboost==2.1.4
7 joblib==1.4.2
8 Werkzeug==2.2.2
9 flask==2.2.2
10 gunicorn==20.1.0
11 requests==2.32.3
12 uvicorn[standard]
13 streamlit==1.43.2
14
```

This file is meant for personal use by cmurray4492@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Deployment - Backend (Flask API - Dockerfile)

The screenshot shows the Hugging Face interface for a Space named 'SuperKart_API'. The 'Files' tab is selected, displaying a Dockerfile. The Dockerfile is a multi-stage build using Python 3.9-slim as the base image. It sets the working directory to /app, copies the current directory's contents into it, and installs dependencies from a requirements.txt file. The container is configured to run the application using Gunicorn with 4 worker processes, binding to port 7860 on all network interfaces. The CMD is set to run gunicorn with the specified configuration and the app name 'superkart_api'.

Hugging Face Search models, datasets, users...

Models Datasets Spaces Posts Docs Enterprise Pricing

Spaces: like 0 Running Logs

App Files Community Settings

main SuperKart_API / Dockerfile

Upload folder using huggingface_hub 2c68abb VERIFIED 7 days ago

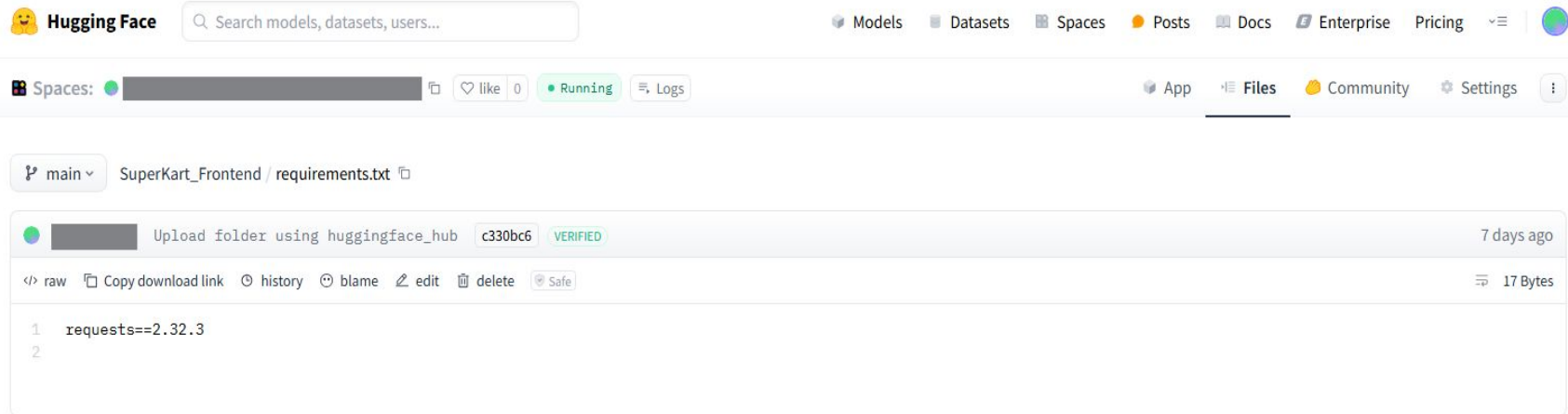
raw Copy download link history blame edit delete Safe 716 Bytes

```
1 FROM python:3.9-slim
2
3 # Set the working directory inside the container
4 WORKDIR /app
5
6 # Copy all files from the current directory to the container's working directory
7 COPY . .
8
9 # Install dependencies from the requirements file without using cache to reduce image size
10 RUN pip install --no-cache-dir --upgrade -r requirements.txt
11
12 # Define the command to start the application using Gunicorn with 4 worker processes
13 # - '-w 4': Uses 4 worker processes for handling requests
14 # - '-b 0.0.0.0:7860': Binds the server to port 7860 on all network interfaces
15 # - 'app:app': Runs the Flask app (assuming 'app.py' contains the Flask instance named 'app')
16 CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:7860", "app:superkart_api"]
17
```

This file is meant for personal use by cmurray4492@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.



Deployment - Frontend (Streamlit - requirements.txt)



The screenshot shows the Hugging Face interface for a Space named "SuperKart_Frontend". The file "requirements.txt" is open, showing the content "requests==2.32.3". The interface includes a search bar, navigation tabs (Models, Datasets, Spaces, Posts, Docs, Enterprise, Pricing), and a file explorer. The file is uploaded from "huggingface_hub" and is verified.

Hugging Face Search models, datasets, users...

Models Datasets Spaces Posts Docs Enterprise Pricing

Spaces:   like 0 Running Logs

App Files Community Settings

main SuperKart_Frontend / requirements.txt

Upload folder using huggingface_hub c330bc6 VERIFIED 7 days ago

raw Copy download link history blame edit delete Safe 17 Bytes

```
1 requests==2.32.3
2
```

Note : Since we are deploying it on the Streamlit space, we don't need to specify streamlit library.



Happy Learning !

